```
# ============================================================
# Full Stack
# ============================================================

# ------------------------------------------------------------
# Application Load Balancer
# ------------------------------------------------------------

resource "aws_lb" "app" {
  name               = "${var.project_name}-${var.environment}-alb"
  internal           = false
  load_balancer_type = "application"
  security_groups    = [aws_security_group.alb.id]
  subnets            = aws_subnet.public[*].id

  enable_deletion_protection = var.environment == "production"

  access_logs {
    bucket  = aws_s3_bucket.logs.bucket
    prefix  = "alb"
    enabled = true
  }

  tags = {
    Name = "${var.project_name}-${var.environment}-alb"
  }
}

resource "aws_lb_target_group" "app" {
  name        = "${var.project_name}-${var.environment}-tg"
  port        = 8080
  protocol    = "HTTP"
  vpc_id      = aws_vpc.main.id
  target_type = "ip"

  health_check {
    enabled             = true
    healthy_threshold   = 2
    unhealthy_threshold = 3
    timeout             = 5
    interval            = 30
    path                = "/health"
    matcher             = "200"
  }

  deregistration_delay = 30

  tags = {
    Name = "${var.project_name}-${var.environment}-tg"
  }
}

resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.app.arn
  port              = 80
  protocol          = "HTTP"

  default_action {
    type = "redirect"

    redirect {
      port        = "443"
      protocol    = "HTTPS"
      status_code = "HTTP_301"
    }
  }
}

resource "aws_lb_listener" "https" {
  load_balancer_arn = aws_lb.app.arn
  port              = 443
  protocol          = "HTTPS"
  ssl_policy        = "ELBSecurityPolicy-TLS13-1-2-2021-06"
  certificate_arn   = aws_acm_certificate.app.arn

  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.app.arn
  }
}

# ------------------------------------------------------------
# ECS Cluster
# ------------------------------------------------------------

resource "aws_ecs_cluster" "main" {
  name = "${var.project_name}-${var.environment}-cluster"

  setting {
    name  = "containerInsights"
    value = "enabled"
  }

  tags = {
    Name = "${var.project_name}-${var.environment}-cluster"
  }
}

resource "aws_ecs_cluster_capacity_providers" "main" {
  cluster_name = aws_ecs_cluster.main.name

  capacity_providers = ["FARGATE", "FARGATE_SPOT"]

  default_capacity_provider_strategy {
    base              = 1
    weight            = 100
    capacity_provider = "FARGATE"
  }
}

# ------------------------------------------------------------
# ECS Task Definition
# ------------------------------------------------------------

resource "aws_ecs_task_definition" "app" {
  family                   = "${var.project_name}-${var.environment}-app"
  network_mode             = "awsvpc"
  requires_compatibilities = ["FARGATE"]
  cpu                      = var.task_cpu
  memory                   = var.task_memory
  execution_role_arn       = aws_iam_role.ecs_execution.arn
  task_role_arn            = aws_iam_role.ecs_task.arn

  container_definitions = jsonencode([
    {
      name      = "app"
      image     = "${aws_ecr_repository.app.repository_url}:${var.image_tag}"
      essential = true

      portMappings = [
        {
          containerPort = 8080
          protocol      = "tcp"
        }
      ]

      environment = [
        { name = "ENVIRONMENT", value = var.environment },
        { name = "PORT",        value = "8080" }
      ]

      secrets = [
        { name = "DB_PASSWORD", valueFrom = aws_secretsmanager_secret.db_password.arn },
        { name = "API_KEY",     valueFrom = aws_secretsmanager_secret.api_key.arn }
      ]

      logConfiguration = {
        logDriver = "awslogs"
        options = {
          "awslogs-group"         = aws_cloudwatch_log_group.app.name
          "awslogs-region"        = var.aws_region
          "awslogs-stream-prefix" = "app"
        }
      }

      healthCheck = {
        command     = ["CMD-SHELL", "curl -f http://localhost:8080/health || exit 1"]
        interval    = 30
        timeout     = 5
        retries     = 3
        startPeriod = 60
      }
    }
  ])
}

# ------------------------------------------------------------
# ECS Service
# ------------------------------------------------------------

resource "aws_ecs_service" "app" {
  name            = "${var.project_name}-${var.environment}-service"
  cluster         = aws_ecs_cluster.main.id
  task_definition = aws_ecs_task_definition.app.arn
  desired_count   = var.desired_count
  launch_type     = "FARGATE"

  network_configuration {
    subnets          = aws_subnet.private[*].id
    security_groups  = [aws_security_group.app.id]
    assign_public_ip = false
  }

  load_balancer {
    target_group_arn = aws_lb_target_group.app.arn
    container_name   = "app"
    container_port   = 8080
  }

  deployment_controller {
    type = "ECS"
  }

  deployment_circuit_breaker {
    enable   = true
    rollback = true
  }

  lifecycle {
    ignore_changes = [desired_count]
  }
}

# ------------------------------------------------------------
# Auto Scaling
# ------------------------------------------------------------

resource "aws_appautoscaling_target" "ecs" {
  max_capacity       = var.max_count
  min_capacity       = var.min_count
  resource_id        = "service/${aws_ecs_cluster.main.name}/${aws_ecs_service.app.name}"
  scalable_dimension = "ecs:service:DesiredCount"
  service_namespace  = "ecs"
}

resource "aws_appautoscaling_policy" "cpu" {
  name               = "${var.project_name}-cpu-scaling"
  policy_type        = "TargetTrackingScaling"
  resource_id        = aws_appautoscaling_target.ecs.resource_id
  scalable_dimension = aws_appautoscaling_target.ecs.scalable_dimension
  service_namespace  = aws_appautoscaling_target.ecs.service_namespace

  target_tracking_scaling_policy_configuration {
    predefined_metric_specification {
      predefined_metric_type = "ECSServiceAverageCPUUtilization"
    }
    target_value       = 70.0
    scale_in_cooldown  = 300
    scale_out_cooldown = 60
  }
}

resource "aws_appautoscaling_policy" "memory" {
  name               = "${var.project_name}-memory-scaling"
  policy_type        = "TargetTrackingScaling"
  resource_id        = aws_appautoscaling_target.ecs.resource_id
  scalable_dimension = aws_appautoscaling_target.ecs.scalable_dimension
  service_namespace  = aws_appautoscaling_target.ecs.service_namespace

  target_tracking_scaling_policy_configuration {
    predefined_metric_specification {
      predefined_metric_type = "ECSServiceAverageMemoryUtilization"
    }
    target_value       = 80.0
    scale_in_cooldown  = 300
    scale_out_cooldown = 60
  }
}

# ------------------------------------------------------------
# RDS
# ------------------------------------------------------------

resource "aws_db_subnet_group" "main" {
  name       = "${var.project_name}-${var.environment}-db-subnet"
  subnet_ids = aws_subnet.private[*].id

  tags = {
    Name = "${var.project_name}-${var.environment}-db-subnet"
  }
}

resource "aws_db_instance" "main" {
  identifier     = "${var.project_name}-${var.environment}-db"
  engine         = "postgres"
  engine_version = "15.4"
  instance_class = var.db_instance_class

  allocated_storage     = 20
  max_allocated_storage = 100
  storage_type          = "gp3"
  storage_encrypted     = true

  db_name  = var.db_name
  username = var.db_username
  password = random_password.db.result

  db_subnet_group_name   = aws_db_subnet_group.main.name
  vpc_security_group_ids = [aws_security_group.db.id]

  backup_retention_period = var.environment == "production" ? 7 : 1
  backup_window           = "03:00-04:00"
  maintenance_window      = "Mon:04:00-Mon:05:00"

  multi_az            = var.environment == "production"
  deletion_protection = var.environment == "production"
  skip_final_snapshot = var.environment != "production"

  performance_insights_enabled = true

  tags = {
    Name = "${var.project_name}-${var.environment}-db"
  }
}

# ------------------------------------------------------------
# ElastiCache
# ------------------------------------------------------------

resource "aws_elasticache_subnet_group" "main" {
  name       = "${var.project_name}-${var.environment}-cache-subnet"
  subnet_ids = aws_subnet.private[*].id
}

resource "aws_elasticache_replication_group" "main" {
  replication_group_id       = "${var.project_name}-${var.environment}-redis"
  description                = "Redis cluster for ${var.project_name}"
  engine                     = "redis"
  engine_version             = "7.0"
  node_type                  = var.cache_node_type
  num_cache_clusters         = var.environment == "production" ? 2 : 1
  port                       = 6379
  subnet_group_name          = aws_elasticache_subnet_group.main.name
  security_group_ids         = [aws_security_group.cache.id]
  automatic_failover_enabled = var.environment == "production"
  at_rest_encryption_enabled = true
  transit_encryption_enabled = true
  auth_token                 = random_password.cache.result

  tags = {
    Name = "${var.project_name}-${var.environment}-redis"
  }
}

# ------------------------------------------------------------
# Secrets Manager
# ------------------------------------------------------------

resource "random_password" "db" {
  length  = 32
  special = false
}

resource "random_password" "cache" {
  length  = 32
  special = false
}

resource "aws_secretsmanager_secret" "db_password" {
  name = "${var.project_name}/${var.environment}/db-password"
}

resource "aws_secretsmanager_secret_version" "db_password" {
  secret_id     = aws_secretsmanager_secret.db_password.id
  secret_string = random_password.db.result
}

resource "aws_secretsmanager_secret" "api_key" {
  name = "${var.project_name}/${var.environment}/api-key"
}

resource "aws_secretsmanager_secret_version" "api_key" {
  secret_id     = aws_secretsmanager_secret.api_key.id
  secret_string = var.api_key
}

# ------------------------------------------------------------
# CloudWatch Logs
# ------------------------------------------------------------

resource "aws_cloudwatch_log_group" "app" {
  name              = "/ecs/${var.project_name}-${var.environment}"
  retention_in_days = var.environment == "production" ? 30 : 7
}

# ------------------------------------------------------------
# ECR
# ------------------------------------------------------------

resource "aws_ecr_repository" "app" {
  name                 = "${var.project_name}-${var.environment}"
  image_tag_mutability = "MUTABLE"

  image_scanning_configuration {
    scan_on_push = true
  }

  encryption_configuration {
    encryption_type = "AES256"
  }
}

resource "aws_ecr_lifecycle_policy" "app" {
  repository = aws_ecr_repository.app.name

  policy = jsonencode({
    rules = [
      {
        rulePriority = 1
        description  = "Keep last 10 images"
        selection = {
          tagStatus     = "any"
          countType     = "imageCountMoreThan"
          countNumber   = 10
        }
        action = {
          type = "expire"
        }
      }
    ]
  })
}

# ------------------------------------------------------------
# IAM
# ------------------------------------------------------------

resource "aws_iam_role" "ecs_execution" {
  name = "${var.project_name}-${var.environment}-ecs-execution"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Action = "sts:AssumeRole"
        Effect = "Allow"
        Principal = {
          Service = "ecs-tasks.amazonaws.com"
        }
      }
    ]
  })
}

resource "aws_iam_role_policy_attachment" "ecs_execution" {
  role       = aws_iam_role.ecs_execution.name
  policy_arn = "arn:aws:iam::aws:policy/service-role/AmazonECSTaskExecutionRolePolicy"
}

resource "aws_iam_role_policy" "ecs_execution_secrets" {
  name = "secrets-access"
  role = aws_iam_role.ecs_execution.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Action = [
          "secretsmanager:GetSecretValue"
        ]
        Effect = "Allow"
        Resource = [
          aws_secretsmanager_secret.db_password.arn,
          aws_secretsmanager_secret.api_key.arn
        ]
      }
    ]
  })
}

resource "aws_iam_role" "ecs_task" {
  name = "${var.project_name}-${var.environment}-ecs-task"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Action = "sts:AssumeRole"
        Effect = "Allow"
        Principal = {
          Service = "ecs-tasks.amazonaws.com"
        }
      }
    ]
  })
}

resource "aws_iam_role_policy" "ecs_task_s3" {
  name = "s3-access"
  role = aws_iam_role.ecs_task.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Action = [
          "s3:GetObject",
          "s3:PutObject",
          "s3:DeleteObject"
        ]
        Effect = "Allow"
        Resource = [
          "${aws_s3_bucket.uploads.arn}/*"
        ]
      }
    ]
  })
}

# ------------------------------------------------------------
# S3
# ------------------------------------------------------------

resource "aws_s3_bucket" "uploads" {
  bucket = "${var.project_name}-${var.environment}-uploads"
}

resource "aws_s3_bucket_versioning" "uploads" {
  bucket = aws_s3_bucket.uploads.id
  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "uploads" {
  bucket = aws_s3_bucket.uploads.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

resource "aws_s3_bucket_public_access_block" "uploads" {
  bucket = aws_s3_bucket.uploads.id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

resource "aws_s3_bucket" "logs" {
  bucket = "${var.project_name}-${var.environment}-logs"
}

resource "aws_s3_bucket_lifecycle_configuration" "logs" {
  bucket = aws_s3_bucket.logs.id

  rule {
    id     = "expire-old-logs"
    status = "Enabled"

    expiration {
      days = 90
    }
  }
}
```

```
message Arm {
  oneof type {
    Servo servo = 1;
    Stepper stepper = 2;
  }
}

enum Interpolation {
  INTERPOLATION_UNSPECIFIED = 0;
  INTERPOLATION_LINEAR = 1;
  INTERPOLATION_CUBIC = 2;
```

```
```

```
```

```
```

```
```

```
# Generated by Django 4.2.11 on 2024-04-15 19:47

from django.db import migrations, models


class Migration(migrations.Migration):

    initial = True

    dependencies = [
    ]

    operations = [
        migrations.CreateModel(
            name="Question",
            fields=[
                ("id", models.BigAutoField(auto_created=True, primary_key=True, serialize=False, verbose_name="ID")),
                ("question_text", models.CharField(max_length=200)),
                ("pub_date", models.DateTimeField(verbose_name="date published")),
            ],
        ),
        migrations.CreateModel(
            name="Choice",
            fields=[
                ("id", models.BigAutoField(auto_created=True, primary_key=True, serialize=False, verbose_name="ID")),
                ("choice_text", models.CharField(max_length=200)),
                ("votes", models.IntegerField(default=0)),
                (
                    "question",
                    models.ForeignKey(on_delete=django.db.models.deletion.CASCADE, to="polls.question"),
                ),
            ],
        ),
    ]
```

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
  labels:
    app: nginx
    tier: frontend
    environment: production
spec:
  type: NodePort
  selector:
    app: nginx
    tier: frontend
  ports:
  - protocol: TCP
    port: 80
    targetPort: 80
    nodePort: 30080
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-config
  namespace: default
data:
  nginx.conf: |
    user nginx;
    worker_processes auto;
    error_log /var/log/nginx/error.log warn;
    pid /var/run/nginx.pid;
    events {
        worker_connections 1024;
        multi_accept on;
        use epoll;
    }
    http {
        include /etc/nginx/mime.types;
        default_type application/octet-stream;
        log_format main '$remote_addr - $remote_user [$time_local] "$request" '
                        '$status $body_bytes_sent "$http_referer" '
                        '"$http_user_agent" "$http_x_forwarded_for"';
        access_log /var/log/nginx/access.log main;
        sendfile on;
        tcp_nopush on;
        tcp_nodelay on;
        keepalive_timeout 65;
        types_hash_max_size 2048;
        server_tokens off;
        gzip on;
        gzip_vary on;
        gzip_proxied any;
        gzip_comp_level 6;
        gzip_types text/plain text/css application/json application/javascript text/xml application/xml;
        server {
            listen 80;
            server_name localhost;
            location / {
                root /usr/share/nginx/html;
                index index.html index.htm;
                try_files $uri $uri/ =404;
            }
            location /health {
                access_log off;
                return 200 "healthy\n";
                add_header Content-Type text/plain;
            }
            location /nginx_status {
                stub_status on;
                access_log off;
                allow 127.0.0.1;
                deny all;
            }
            error_page 500 502 503 504 /50x.html;
            location = /50x.html {
                root /usr/share/nginx/html;
            }
        }
    }
---
apiVersion: v1
kind: Secret
metadata:
  name: app-secrets
  namespace: default
type: Opaque
stringData:
  DB_PASSWORD: "supersecretpassword123"
  API_KEY: "abcdef1234567890"
  JWT_SECRET: "my-jwt-secret-key-for-signing-tokens"
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data-pvc
  namespace: default
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: standard
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-deployment
  namespace: default
  labels:
    app: myapp
    version: v1.2.3
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
        version: v1.2.3
    spec:
      containers:
      - name: myapp
        image: myapp:1.2.3
        ports:
        - containerPort: 8080
          name: http
        env:
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: app-secrets
              key: DB_PASSWORD
        - name: API_KEY
          valueFrom:
            secretKeyRef:
              name: app-secrets
              key: API_KEY
        - name: JWT_SECRET
          valueFrom:
            secretKeyRef:
              name: app-secrets
              key: JWT_SECRET
        volumeMounts:
        - name: data
          mountPath: /data
        - name: config
          mountPath: /etc/myapp
          readOnly: true
        resources:
          requests:
            memory: "256Mi"
            cpu: "250m"
          limits:
            memory: "512Mi"
            cpu: "500m"
        livenessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 30
          periodSeconds: 10
        readinessProbe:
          httpGet:
            path: /ready
            port: 8080
          initialDelaySeconds: 5
          periodSeconds: 5
      volumes:
      - name: data
        persistentVolumeClaim:
          claimName: data-pvc
      - name: config
        configMap:
          name: app-config
---
apiVersion: v1
kind: Service
metadata:
  name: app-service
  namespace: default
spec:
  type: ClusterIP
  selector:
    app: myapp
  ports:
  - port: 80
    targetPort: 8080
    protocol: TCP
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  namespace: default
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
spec:
  tls:
  - hosts:
    - myapp.example.com
    secretName: tls-secret
  rules:
  - host: myapp.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: app-service
            port:
              number: 80
---
apiVersion: batch/v1
kind: CronJob
metadata:
  name: backup-job
  namespace: default
spec:
  schedule: "0 2 * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
          - name: backup
            image: backup-tool:latest
            command:
            - /bin/sh
            - -c
            - |
              tar -czf /backup/data-$(date +%Y%m%d).tar.gz /data
              find /backup -mtime +7 -delete
            volumeMounts:
            - name: data
              mountPath: /data
              readOnly: true
            - name: backup
              mountPath: /backup
          volumes:
          - name: data
            persistentVolumeClaim:
              claimName: data-pvc
          - name: backup
            persistentVolumeClaim:
              claimName: backup-pvc
          restartPolicy: OnFailure
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-sa
  namespace: default
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: default
  name: app-role
rules:
- apiGroups: [""]
  resources: ["pods", "services"]
  verbs: ["get", "list", "watch"]
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["get", "list", "watch", "update"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: app-rolebinding
  namespace: default
subjects:
- kind: ServiceAccount
  name: app-sa
  namespace: default
roleRef:
  kind: Role
  name: app-role
  apiGroup: rbac.authorization.k8s.io
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: app-hpa
  namespace: default
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: app-deployment
  minReplicas: 3
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
  - type: Resource
    resource:
      name: memory
      target:
        type: Utilization
        averageUtilization: 80
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: app-network-policy
  namespace: default
spec:
  podSelector:
    matchLabels:
      app: myapp
  policyTypes:
  - Ingress
  - Egress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
    ports:
    - protocol: TCP
      port: 8080
  egress:
  - to:
    - podSelector:
        matchLabels:
          app: database
    ports:
    - protocol: TCP
      port: 5432
  - to:
    - namespaceSelector: {}
    ports:
    - protocol: TCP
      port: 53
    - protocol: UDP
      port: 53
---
apiVersion: v1
kind: PersistentVolume
metadata:
  name: local-pv
  labels:
    type: local
spec:
  capacity:
    storage: 100Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: local-storage
  local:
    path: /mnt/disks/ssd1
  nodeAffinity:
    required:
      nodeSelectorTerms:
      - matchExpressions:
        - key: kubernetes.io/hostname
          operator: In
          values:
          - node-1
---
apiVersion: v1
kind: Pod
metadata:
  name: debug-pod
  namespace: default
  labels:
    app: debug
spec:
  containers:
  - name: debug
    image: busybox:1.36
    command: ["sleep", "3600"]
    resources:
      requests:
        memory: "64Mi"
        cpu: "50m"
      limits:
        memory: "128Mi"
        cpu: "100m"
  restartPolicy: Never
---
apiVersion: v1
kind: ResourceQuota
metadata:
  name: namespace-quota
  namespace: default
spec:
  hard:
    requests.cpu: "4"
    requests.memory: 8Gi
    limits.cpu: "8"
    limits.memory: 16Gi
    pods: "20"
    services: "10"
    persistentvolumeclaims: "5"
---
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: default
spec:
  limits:
  - default:
      cpu: 500m
      memory: 512Mi
    defaultRequest:
      cpu: 250m
      memory: 256Mi
    type: Container
---
apiVersion: v1
kind: Service
metadata:
  name: redis-service
  namespace: default
spec:
  type: ClusterIP
  selector:
    app: redis
  ports:
  - port: 6379
    targetPort: 6379
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: redis-cluster
  namespace: default
spec:
  serviceName: redis-service
  replicas: 3
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
      - name: redis
        image: redis:7.2-alpine
        ports:
        - containerPort: 6379
        volumeMounts:
        - name: redis-data
          mountPath: /data
        command:
        - redis-server
        - --appendonly yes
        - --maxmemory 256mb
        - --maxmemory-policy allkeys-lru
  volumeClaimTemplates:
  - metadata:
      name: redis-data
    spec:
      accessModes: ["ReadWriteOnce"]
      resources:
        requests:
          storage: 1Gi
---
apiVersion: v1
kind: Service
metadata:
  name: postgres-service
  namespace: default
spec:
  type: ClusterIP
  selector:
    app: postgres
  ports:
  - port: 5432
    targetPort: 5432
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
  namespace: default
spec:
  serviceName: postgres-service
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
      - name: postgres
        image: postgres:16-alpine
        ports:
        - containerPort: 5432
        env:
        - name: POSTGRES_DB
          value: appdb
        - name: POSTGRES_USER
          valueFrom:
            secretKeyRef:
              name: db-credentials
              key: username
        - name: POSTGRES_PASSWORD
          valueFrom:
            secretKeyRef:
              name: db-credentials
              key: password
        - name: PGDATA
          value: /var/lib/postgresql/data/pgdata
        volumeMounts:
        - name: postgres-data
          mountPath: /var/lib/postgresql/data
        resources:
          requests:
            memory: "512Mi"
            cpu: "500m"
          limits:
            memory: "1Gi"
            cpu: "1000m"
  volumeClaimTemplates:
  - metadata:
      name: postgres-data
    spec:
      accessModes: ["ReadWriteOnce"]
      resources:
        requests:
          storage: 20Gi
---
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
  namespace: default
type: Opaque
stringData:
  username: appuser
  password: "dbpassword456"
---
apiVersion: v1
kind: Service
metadata:
  name: rabbitmq-service
  namespace: default
spec:
  type: ClusterIP
  selector:
    app: rabbitmq
  ports:
  - port: 5672
    targetPort: 5672
  - port: 15672
    targetPort: 15672
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: rabbitmq
  namespace: default
spec:
  replicas: 1
  selector:
    matchLabels:
      app: rabbitmq
  template:
    metadata:
      labels:
        app: rabbitmq
    spec:
      containers:
      - name: rabbitmq
        image: rabbitmq:3.12-management-alpine
        ports:
        - containerPort: 5672
        - containerPort: 15672
        env:
        - name: RABBITMQ_DEFAULT_USER
          value: admin
        - name: RABBITMQ_DEFAULT_PASS
          value: "rabbitpassword789"
        volumeMounts:
        - name: rabbitmq-data
          mountPath: /var/lib/rabbitmq
      volumes:
      - name: rabbitmq-data
        emptyDir: {}
---
apiVersion: v1
kind: Service
metadata:
  name: elasticsearch-service
  namespace: default
spec:
  type: ClusterIP
  selector:
    app: elasticsearch
  ports:
  - port: 9200
    targetPort: 9200
  - port: 9300
    targetPort: 9300
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: elasticsearch
  namespace: default
spec:
  serviceName: elasticsearch-service
  replicas: 1
  selector:
    matchLabels:
      app: elasticsearch
  template:
    metadata:
      labels:
        app: elasticsearch
    spec:
      containers:
      - name: elasticsearch
        image: elasticsearch:8.11.0
        ports:
        - containerPort: 9200
        - containerPort: 9300
        env:
        - name: discovery.type
          value: single-node
        - name: ES_JAVA_OPTS
          value: "-Xms512m -Xmx512m"
        - name: xpack.security.enabled
          value: "false"
        volumeMounts:
        - name: es-data
          mountPath: /usr/share/elasticsearch/data
        resources:
          requests:
            memory: "1Gi"
            cpu: "500m"
          limits:
            memory: "2Gi"
            cpu: "1000m"
  volumeClaimTemplates:
  - metadata:
      name: es-data
    spec:
      accessModes: ["ReadWriteOnce"]
      resources:
        requests:
          storage: 50Gi
---
apiVersion: v1
kind: Service
metadata:
  name: kibana-service
  namespace: default
spec:
  type: ClusterIP
  selector:
    app: kibana
  ports:
  - port: 5601
    targetPort: 5601
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kibana
  namespace: default
spec:
  replicas: 1
  selector:
    matchLabels:
      app: kibana
  template:
    metadata:
      labels:
        app: kibana
    spec:
      containers:
      - name: kibana
        image: kibana:8.11.0
        ports:
        - containerPort: 5601
        env:
        - name: ELASTICSEARCH_HOSTS
          value: '["http://elasticsearch-service:9200"]'
        - name: SERVER_NAME
          value: kibana.example.com
        - name: SERVER_BASEPATH
          value: /kibana
        - name: SERVER_REWRITEBASEPATH
          value: "true"
        - name: XPACK_SECURITY_ENABLED
          value: "false"
        resources:
          requests:
            memory: "512Mi"
            cpu: "250m"
          limits:
            memory: "1Gi"
            cpu: "500m"
---
apiVersion: v1
kind: Service
metadata:
  name: grafana-service
  namespace: default
spec:
  type: ClusterIP
  selector:
    app: grafana
  ports:
  - port: 3000
    targetPort: 3000
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: grafana
  namespace: default
spec:
  replicas: 1
  selector:
    matchLabels:
      app: grafana
  template:
    metadata:
      labels:
        app: grafana
    spec:
      containers:
      - name: grafana
        image: grafana/grafana:10.2.0
        ports:
        - containerPort: 3000
        env:
        - name: GF_SECURITY_ADMIN_PASSWORD
          value: "grafanapassword"
        - name: GF_SERVER_ROOT_URL
          value: "%(protocol)s://%(domain)s/grafana"
        - name: GF_SERVER_SERVE_FROM_SUB_PATH
          value: "true"
        volumeMounts:
        - name: grafana-data
          mountPath: /var/lib/grafana
      volumes:
      - name: grafana-data
        emptyDir: {}
---
apiVersion: v1
kind: Service
metadata:
  name: prometheus-service
  namespace: default
spec:
  type: ClusterIP
  selector:
    app: prometheus
  ports:
  - port: 9090
    targetPort: 9090
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: prometheus
  namespace: default
spec:
  replicas: 1
  selector:
    matchLabels:
      app: prometheus
  template:
    metadata:
      labels:
        app: prometheus
    spec:
      containers:
      - name: prometheus
        image: prometheus:v2.48.0
        ports:
        - containerPort: 9090
        volumeMounts:
        - name: prometheus-config
          mountPath: /etc/prometheus
        - name: prometheus-data
          mountPath: /prometheus
        args:
        - --config.file=/etc/prometheus/prometheus.yml
        - --storage.tsdb.path=/prometheus
        - --web.console.libraries=/usr/share/prometheus/console_libraries
        - --web.console.templates=/usr/share/prometheus/consoles
      volumes:
      - name: prometheus-config
        configMap:
          name: prometheus-config
      - name: prometheus-data
        emptyDir: {}
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: prometheus-config
  namespace: default
data:
  prometheus.yml: |
    global:
      scrape_interval: 15s
      evaluation_interval: 15s
    scrape_configs:
      - job_name: 'prometheus'
        static_configs:
          - targets: ['localhost:9090']
      - job_name: 'kubernetes-pods'
        kubernetes_sd_configs:
          - role: pod
        relabel_configs:
          - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
            action: keep
            regex: true
---
apiVersion: v1
kind: Service
metadata:
  name: minio-service
  namespace: default
spec:
  type: ClusterIP
  selector:
    app: minio
  ports:
  - port: 9000
    targetPort: 9000
  - port: 9001
    targetPort: 9001
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: minio
  namespace: default
spec:
  replicas: 1
  selector:
    matchLabels:
      app: minio
  template:
    metadata:
      labels:
        app: minio
    spec:
      containers:
      - name: minio
        image: minio/minio:RELEASE.2023-12-02T10-51-33Z
        ports:
        - containerPort: 9000
        - containerPort: 9001
        env:
        - name: MINIO_ROOT_USER
          value: minioadmin
        - name: MINIO_ROOT_PASSWORD
          value: "miniopassword123"
        volumeMounts:
        - name: minio-data
          mountPath: /data
        command:
        - minio
        - server
        - /data
        - --console-address
        - ":9001"
      volumes:
      - name: minio-data
        emptyDir: {}
---
apiVersion: v1
kind: Service
metadata:
  name: vault-service
  namespace: default
spec:
  type: ClusterIP
  selector:
    app: vault
  ports:
  - port: 8200
    targetPort: 8200
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vault
  namespace: default
spec:
  replicas: 1
  selector:
    matchLabels:
      app: vault
  template:
    metadata:
      labels:
        app: vault
    spec:
      containers:
      - name: vault
        image: hashicorp/vault:1.15.2
        ports:
        - containerPort: 8200
        env:
        - name: VAULT_DEV_ROOT_TOKEN_ID
          value: "vaultroottoken"
        - name: VAULT_DEV_LISTEN_ADDRESS
          value: "0.0.0.0:8200"
        - name: VAULT_ADDR
          value: "http://127.0.0.1:8200"
        volumeMounts:
        - name: vault-data
          mountPath: /vault/data
        - name: vault-config
          mountPath: /vault/config
        args:
        - server
        - -dev
      volumes:
      - name: vault-data
        emptyDir: {}
      - name: vault-config
        emptyDir: {}
---
apiVersion: v1
kind: Service
metadata:
  name: consul-service
  namespace: default
spec:
  type: ClusterIP
  selector:
    app: consul
  ports:
  - port: 8500
    targetPort: 8500
  - port: 8600
    targetPort: 8600
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: consul
  namespace: default
spec:
  serviceName: consul-service
  replicas: 1
  selector:
    matchLabels:
      app: consul
  template:
    metadata:
      labels:
        app: consul
    spec:
      containers:
      - name: consul
        image: consul:1.17.0
        ports:
        - containerPort: 8500
        - containerPort: 8600
        env:
        - name: CONSUL_BIND_INTERFACE
          value: eth0
        volumeMounts:
        - name: consul-data
          mountPath: /consul/data
        args:
        - agent
        - -server
        - -bootstrap-expect=1
        - -ui
        - -client=0.0.0.0
  volumeClaimTemplates:
  - metadata:
      name: consul-data
    spec:
      accessModes: ["ReadWriteOnce"]
      resources:
        requests:
          storage: 5Gi
---
apiVersion: v1
kind: Service
metadata:
  name: jenkins-service
  namespace: default
spec:
  type: ClusterIP
  selector:
    app: jenkins
  ports:
  - port: 8080
    targetPort: 8080
  - port: 50000
    targetPort: 50000
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: jenkins
  namespace: default
spec:
  serviceName: jenkins-service
  replicas: 1
  selector:
    matchLabels:
      app: jenkins
  template:
    metadata:
      labels:
        app: jenkins
    spec:
      containers:
      - name: jenkins
        image: jenkins/jenkins:lts-jdk17
        ports:
        - containerPort: 8080
        - containerPort: 50000
        env:
        - name: JAVA_OPTS
          value: "-Xmx1024m -Xms512m"
        volumeMounts:
        - name: jenkins-home
          mountPath: /var/jenkins_home
        resources:
          requests:
            memory: "1Gi"
            cpu: "500m"
          limits:
            memory: "2Gi"
            cpu: "1000m"
  volumeClaimTemplates:
  - metadata:
      name: jenkins-home
    spec:
      accessModes: ["ReadWriteOnce"]
      resources:
        requests:
          storage: 20Gi
---
apiVersion: v1
kind: Service
metadata:
  name: sonarqube-service
  namespace: default
spec:
  type: ClusterIP
  selector:
    app: sonarqube
  ports:
  - port: 9000
    targetPort: 9000
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sonarqube
  namespace: default
spec:
  replicas: 1
  selector:
    matchLabels:
      app: sonarqube
  template:
    metadata:
      labels:
        app: sonarqube
    spec:
      containers:
      - name: sonarqube
        image: sonarqube:10.3-community
        ports:
        - containerPort: 9000
        env:
        - name: SONAR_JDBC_URL
          value: jdbc:postgresql://postgres-service:5432/sonarqube
        - name: SONAR_JDBC_USERNAME
          value: sonaruser
        - name: SONAR_JDBC_PASSWORD
          value: "sonarpassword"
        volumeMounts:
        - name: sonarqube-data
          mountPath: /opt/sonarqube/data
        - name: sonarqube-extensions
          mountPath: /opt/sonarqube/extensions
      volumes:
      - name: sonarqube-data
        emptyDir: {}
      - name: sonarqube-extensions
        emptyDir: {}
---
apiVersion: v1
kind: Service
metadata:
  name: nexus-service
  namespace: default
spec:
  type: ClusterIP
  selector:
    app: nexus
  ports:
  - port: 8081
    targetPort: 8081
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nexus
  namespace: default
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nexus
  template:
    metadata:
      labels:
        app: nexus
    spec:
      containers:
      - name: nexus
        image: sonatype/nexus3:3.62.0
        ports:
        - containerPort: 8081
        volumeMounts:
        - name: nexus-data
          mountPath: /nexus-data
        resources:
          requests:
            memory: "2Gi"
            cpu: "500m"
          limits:
            memory: "4Gi"
            cpu: "1000m"
      volumes:
      - name: nexus-data
        emptyDir: {}
---
apiVersion: v1
kind: Service
metadata:
  name: traefik-service
  namespace: default
spec:
  type: LoadBalancer
  selector:
    app: traefik
  ports:
  - port: 80
    targetPort: 80
  - port: 443
    targetPort: 443
  - port: 8080
    targetPort: 8080
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: traefik
  namespace: default
spec:
  replicas: 1
  selector:
    matchLabels:
      app: traefik
  template:
    metadata:
      labels:
        app: traefik
    spec:
      containers:
      - name: traefik
        image: traefik:v2.10
        ports:
        - containerPort: 80
        - containerPort: 443
        - containerPort: 8080
        volumeMounts:
        - name: traefik-config
          mountPath: /etc/traefik
        args:
        - --api.insecure=true
        - --providers.kubernetes.ingress=true
        - --entrypoints.web.address=:80
        - --entrypoints.websecure.address=:443
      volumes:
      - name: traefik-config
        emptyDir: {}
---
apiVersion: v1
kind: Service
metadata:
  name: mongodb-service
  namespace: default
spec:
  type: ClusterIP
  selector:
    app: mongodb
  ports:
  - port: 27017
    targetPort: 27017
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mongodb
  namespace: default
spec:
  serviceName: mongodb-service
  replicas: 1
  selector:
    matchLabels:
      app: mongodb
  template:
    metadata:
      labels:
        app: mongodb
    spec:
      containers:
      - name: mongodb
        image: mongo:7.0
        ports:
        - containerPort: 27017
        env:
        - name: MONGO_INITDB_ROOT_USERNAME
          value: mongoadmin
        - name: MONGO_INITDB_ROOT_PASSWORD
          value: "mongopassword"
        volumeMounts:
        - name: mongodb-data
          mountPath: /data/db
  volumeClaimTemplates:
  - metadata:
      name: mongodb-data
    spec:
      accessModes: ["ReadWriteOnce"]
      resources:
        requests:
          storage: 20Gi
---
apiVersion: v1
kind: Service
metadata:
  name: memcached-service
  namespace: default
spec:
  type: ClusterIP
  selector:
    app: memcached
  ports:
  - port: 11211
    targetPort: 11211
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: memcached
  namespace: default
spec:
  replicas: 2
  selector:
    matchLabels:
      app: memcached
  template:
    metadata:
      labels:
        app: memcached
    spec:
      containers:
      - name: memcached
        image: memcached:1.6-alpine
        ports:
        - containerPort: 11211
        args:
        - -m 256
        - -c 1024
        - -t 4
---
apiVersion: v1
kind: Service
metadata:
  name: zookeeper-service
  namespace: default
spec:
  type: ClusterIP
  selector:
    app: zookeeper
  ports:
  - port: 2181
    targetPort: 2181
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: zookeeper
  namespace: default
spec:
  serviceName: zookeeper-service
  replicas: 1
  selector:
    matchLabels:
      app: zookeeper
  template:
    metadata:
      labels:
        app: zookeeper
    spec:
      containers:
      - name: zookeeper
        image: zookeeper:3.9
        ports:
        - containerPort: 2181
        env:
        - name: ZOO_MY_ID
          value: "1"
        - name: ZOO_SERVERS
          value: server.1=zookeeper-service:2888:3888
        volumeMounts:
        - name: zookeeper-data
          mountPath: /data
  volumeClaimTemplates:
  - metadata:
      name: zookeeper-data
    spec:
      accessModes: ["ReadWriteOnce"]
      resources:
        requests:
          storage: 5Gi
---
apiVersion: v1
kind: Service
metadata:
  name: kafka-service
  namespace: default
spec:
  type: ClusterIP
  selector:
    app: kafka
  ports:
  - port: 9092
    targetPort: 9092
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: kafka
  namespace: default
spec:
  serviceName: kafka-service
  replicas: 1
  selector:
    matchLabels:
      app: kafka
  template:
    metadata:
      labels:
        app: kafka
    spec:
      containers:
      - name: kafka
        image: confluentinc/cp-kafka:7.5.0
        ports:
        - containerPort: 9092
        env:
        - name: KAFKA_BROKER_ID
          value: "1"
        - name: KAFKA_ZOOKEEPER_CONNECT
          value: zookeeper-service:2181
        - name: KAFKA_ADVERTISED_LISTENERS
          value: PLAINTEXT://kafka-service:9092
        - name: KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR
          value: "1"
        volumeMounts:
        - name: kafka-data
          mountPath: /var/lib/kafka/data
  volumeClaimTemplates:
  - metadata:
      name: kafka-data
    spec:
      accessModes: ["ReadWriteOnce"]
      resources:
        requests:
          storage: 50Gi
```

```yaml
deny_rule:
  - deny_rule
  - deny_rule
deny_rule_deny:
  - deny_rule_deny_rule_deny_rule_deny_rule
  - deny_rule:
      - deny_rule
      - deny_rule
      - deny_rule
      - deny_rule
      - deny_rule_deny_rule_deny_rule_deny_rule_deny_rule_
```

```
configuration_validation_error_handling_test_case>>>
```

```
    admissionregistration: Admission webhook matchConditions now support authorizer.path() and authorizer.resource() functions for path-based authorization checks. (#121234, @benluddy)
  - apiserver: Fixed a panic in the watch cache when processing events with malformed resource versions. (#121567, @jpbetz)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#121890, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#122012, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#122145, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#122289, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#122401, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#122534, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#122678, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#122801, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#122934, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#123067, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#123189, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#123312, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#123445, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#123578, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#123701, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#123834, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#123967, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#124089, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#124212, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#124345, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#124478, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#124601, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#124723, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#124856, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#124989, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#125112, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#125245, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#125378, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#125501, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#125623, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#125746, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#125879, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#126012, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#126135, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#126257, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#126390, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#126523, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#126646, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#126779, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#126912, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#127035, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#127157, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#127280, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#127413, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#127546, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#127669, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#127791, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#127924, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#128057, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#128180, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#128313, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#128446, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#128569, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#128691, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#128814, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#128947, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#129080, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#129203, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#129325, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#129458, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#129591, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#129714, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#129847, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#129980, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#130103, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#130225, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#130348, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#130481, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#130614, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#130737, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#130859, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#130992, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#131125, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#131248, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#131381, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#131514, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#131637, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#131759, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#131882, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#132015, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#132148, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#132271, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#132393, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#132526, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#132659, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#132782, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#132915, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#133048, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#133171, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#133293, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#133416, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#133549, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#133682, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#133805, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#133927, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#134060, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#134193, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#134316, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#134449, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#134582, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#134705, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#134827, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#134950, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#135083, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#135216, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#135339, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#135461, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#135594, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#135727, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#135850, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#135983, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#136116, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#136239, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#136361, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#136484, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#136617, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#136750, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#136873, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#136995, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#137128, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#137261, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#137384, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#137517, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#137650, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#137773, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#137895, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#138018, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#138151, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#138284, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#138407, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#138529, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#138662, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#138795, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#138918, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#139051, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#139184, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#139307, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#139429, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#139552, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#139685, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#139818, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#139941, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#140063, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#140196, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#140329, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#140452, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#140585, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#140718, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#140841, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#140963, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#141086, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#141219, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#141352, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#141475, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#141597, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#141730, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#141863, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#141986, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#142119, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#142252, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#142375, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#142497, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#142620, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#142753, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#142886, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#143009, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#143131, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#143264, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#143397, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#143520, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#143653, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#143786, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#143909, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#144031, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#144154, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#144287, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#144420, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#144543, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#144665, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#144798, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#144931, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#145054, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#145187, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#145320, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#145443, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#145565, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#145688, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#145821, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#145954, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#146077, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#146199, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#146332, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#146465, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#146588, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#146721, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#146854, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#146977, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#147099, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#147222, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#147355, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#147488, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#147611, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#147733, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#147866, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#147999, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#148122, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#148255, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#148388, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#148511, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#148633, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#148756, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#148889, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#149022, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#149145, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#149267, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#149400, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#149533, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#149656, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#149789, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#149922, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#150045, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#150167, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#150290, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#150423, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#150556, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#150679, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#150801, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#150934, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#151067, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#151190, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#151323, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#151456, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#151579, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#151701, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#151824, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#151957, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#152090, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#152213, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#152335, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#152468, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#152601, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#152724, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#152857, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#152990, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#153113, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#153235, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#153358, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#153491, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#153624, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#153747, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#153869, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#154002, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#154135, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#154258, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#154391, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#154524, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#154647, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#154769, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#154892, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#155025, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#155158, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#155281, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#155403, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#155536, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#155669, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#155792, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#155925, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#156058, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#156181, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#156303, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#156426, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#156559, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#156692, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#156815, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#156937, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#157070, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#157203, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#157326, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#157459, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#157592, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#157715, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#157837, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#157960, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#158093, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#158226, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#158349, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#158471, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#158604, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#158737, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#158860, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#158993, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#159126, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#159249, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#159371, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#159494, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#159627, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#159760, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#159883, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#160005, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#160138, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#160271, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#160394, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#160527, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#160660, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#160783, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#160905, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#161028, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#161161, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#161294, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#161417, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#161539, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#161672, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#161805, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#161928, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#162061, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#162194, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#162317, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#162439, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#162562, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#162695, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#162828, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#162951, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#163073, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#163206, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#163339, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#163462, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#163595, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#163728, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#163851, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#163973, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#164096, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#164229, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#164362, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#164485, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#164607, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#164740, @wojtek-t)
  - apiserver: Added support for streaming encoding of large list responses to reduce memory allocations. (#164873, @wojtek-t)
  - apiserver: Improved error messages when admission webhooks return invalid responses. (#164996, @deads2k)
  - apiserver: Fixed a race condition in the API priority and fairness dispatcher that could cause requests to be starved. (#165129, @MikeSpreitzer)
  - apiserver: Added metrics for tracking the number of in-flight requests per flow schema. (#165262, @MikeSpreitzer)
  - apiserver: Fixed a memory leak in the watch cache when clients disconnect abruptly. (#165385, @wojtek-t)
  - apiserver: Improved performance of label selector matching for large numbers of objects. (#165507, @wojtek-t)
  - apiserver: Added support for CEL expressions in admission webhook matchConditions. (#165630, @benluddy)
  - apiserver: Fixed a bug where audit logs could be duplicated when using multiple audit backends. (#165763, @tallclair)
  - apiserver: Improved the efficiency of the watch cache initialization process. (#165896, @wojtek-t)
  - apiserver: Added support for dry-run requests in the API priority and fairness system. (#166019, @MikeSpreitzer)
  - apiserver: Fixed a panic in the admission webhook dispatcher when a webhook returns an empty response. (#166141, @deads2k)
  - apiserver: Improved the performance of the watch cache for clusters with many namespaces. (#166274, @w
```

```
```
```
```

```
# migrate_user_data_to_new_schema.py
# Migrates user records from the old flat schema to the new normalized schema.
# Run with: python migrate_user_data_to_new_schema.py --dry-run

import argparse
import logging
import sys
from datetime import datetime, timezone
from typing import Iterable, Optional

import psycopg2
import psycopg2.extras

BATCH_SIZE = 500
LOG = logging.getLogger("migrate")


def parse_args(argv):
    p = argparse.ArgumentParser(description="Migrate user data to new schema")
    p.add_argument("--dry-run", action="store_true", help="Do not write changes")
    p.add_argument("--source-dsn", required=True)
    p.add_argument("--target-dsn", required=True)
    p.add_argument("--limit", type=int, default=0, help="Stop after N rows (0 = all)")
    p.add_argument("--resume-from-id", type=int, default=0)
    return p.parse_args(argv)


def configure_logging(verbose: bool):
    level = logging.DEBUG if verbose else logging.INFO
    logging.basicConfig(
        level=level,
        format="%(asctime)s %(levelname)s %(name)s: %(message)s",
        stream=sys.stderr,
    )


def fetch_batches(conn, last_id: int, limit: int) -> Iterable[list]:
    """Yield batches of legacy user rows ordered by id."""
    with conn.cursor(cursor_factory=psycopg2.extras.DictCursor) as cur:
        cur.execute(
            "SELECT id, email, full_name, created_at, is_active "
            "FROM legacy_users WHERE id > %s ORDER BY id LIMIT %s",
            (last_id, limit or None),
        )
        while True:
            rows = cur.fetchmany(BATCH_SIZE)
            if not rows:
                return
            yield rows


def transform(row) -> dict:
    """Map a legacy row to the new schema."""
    return {
        "id": row["id"],
        "email": row["email"].strip().lower(),
        "display_name": row["full_name"].strip(),
        "created_at": row["created_at"] or datetime.now(timezone.utc),
        "status": "active" if row["is_active"] else "disabled",
    }


def write_batch(conn, rows: list, dry_run: bool):
    if dry_run:
        LOG.debug("dry-run: would write %d rows", len(rows))
        return
    with conn.cursor() as cur:
        psycopg2.extras.execute_values(
            cur,
            "INSERT INTO users (id, email, display_name, created_at, status) VALUES %s "
            "ON CONFLICT (id) DO UPDATE SET email = EXCLUDED.email, "
            "display_name = EXCLUDED.display_name, status = EXCLUDED.status",
            rows,
        )


def main(argv=None):
    args = parse_args(argv or sys.argv[1:])
    configure_logging(verbose=True)

    src = psycopg2.connect(args.source_dsn)
    dst = psycopg2.connect(args.target_dsn)
    src.set_session(readonly=True)

    total = 0
    last_id = args.resume_from_id
    try:
        for batch in fetch_batches(src, last_id, args.limit):
            rows = [transform(r) for r in batch]
            write_batch(dst, rows, args.dry_run)
            if not args.dry_run:
                dst.commit()
            total += len(rows)
            last_id = batch[-1]["id"]
            LOG.info("migrated %d rows (last id %d)", total, last_id)
            if args.limit and total >= args.limit:
                break
    except Exception:
        dst.rollback()
        LOG.exception("migration failed at id %d", last_id)
        return 1
    finally:
        src.close()
        dst.close()

    LOG.info("done: %d rows", total)
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```